Test Case Correctness & Coverage — Brief

What's built

TestAtlas parses a codebase into a knowledge graph (files, classes, functions, call relations), clusters it into modules, and uses an LLM to generate functional test cases per module (title, steps, expected result, ADO-shape), grounded with real UI strings pulled from the code and tagged with a code-traceability reference. Output is explicitly framed as a first draft for human review, not verified fact.

Problem

Two separate, easily-conflated questions have no built-in answer today:

- **Grounding** — is a given generated test case *true*? It's prose with no runnable assertion, so there's no pass/fail signal, just a plausible-sounding claim.
- **Coverage** — do we have the *right* test cases? Modules are shaped by package/folder structure, not by how a user experiences the product. "Checkout" is likely split across login, search, cart, payment, and address — unrelated modules. Testing each module doesn't tell you "checkout is covered."

Ideas discussed, and verdict

#	Idea	Targets	Verdict
1	Diff generated cases against the existing test suite	Grounding	Different altitude (assertion vs. narrated behavior) — only checks the narrow slice of claims that happen to overlap. Useful as a bolt-on fact-check, not standalone.
2	Generate user stories from code instead of test cases	Coverage	Correct instinct, but deriving stories bottom-up from the same module clustering just relocates the granularity mismatch rather than fixing it.
3	Compare existing (human-written) user stories against the code — % implemented, cross-referenced with existing tests	Coverage	Strongest option — anchors to a real, external unit of scope, so “checkout spans 5 modules” is expected, not a clustering failure. Depends on a story already existing, and needs a new step-to-code mapping that isn’t solved yet.

Recommended approach (two independent tracks)

Track A — Grounding. Layer cheap, deterministic checks before expensive ones:

1. *Citation checking* (do first) — require every step to cite the graph node it came from, then verify programmatically (no LLM) that the node exists and the claimed string is really in its source. Near-free, catches invented facts.
2. *Self-consistency sampling* — generate the same module more than once and flag claims that don’t recur across runs. No oracle needed, but multiplies per-module cost for a signal item 3 usually reaches more cheaply — kept in reserve, not the default.
3. *Separate critic pass* (preferred default after citation checking) — a second, adversarially-framed LLM call over the already-generated case, asked to find unsupported claims and cite contradicting code.

CONFIDENTIAL

4. *Execution/coverage cross-check* — only where a repo happens to be runnable or already has a working suite with coverage data. Optional enhancement, not a dependency — most analyzed repos can't be built/seeded on demand.
5. Treat the result as a **confidence signal**, not a verdict — and once test cases are editable, calibrate that confidence against real accept/edit/reject behavior over time.

Track B — Coverage. Build idea 3 as designed (extend Gap Analysis to story granularity) for repos that have written stories. For the far more common case where none exist: **cluster by entry point / request path instead of by package** — trace the outbound call chain from real entry points (HTTP routes, top-level pages, CLI commands). This groups code the way a request actually flows, not the way folders happen to be organized, with no external document required.

Result: a claim like *"Checkout is 70% implemented, 9 of 12 generated test cases cite verified code with no contradictions, 3 flagged for review"* — materially stronger than an unvetted list, without requiring a runnable environment, an existing suite, or a pre-written story as a hard dependency everywhere.

Challenges either way

- Most target repos can't be built/seeded/run — rules out execution-based checks as a default.
- Extra LLM calls (critic, sampling) multiply the already-visible per-module metered cost.
- Every technique here yields a confidence distribution, not true/false — how that's surfaced to a reviewer without becoming ignorable noise is its own design problem.
- Grounding and coverage are independent failures — fixing one doesn't fix the other; both tracks need to move in parallel.

Next step

Build citation checking first (Track A, item 1) — no new model calls, targets the worst failure mode (invented facts), and is cheap to validate against a few repos by hand before anything else is designed further.

CONFIDENTIAL